big.Little Scheduling assignment

The objective of this assignment is to understand the complexity of scheduling processes on a system with heterogeneous processor cores, namely, ARM's big.LITTLE processor configuration, where High performance (big cores) are coupled with Energy efficient (Little) cores. This design is widely used in mobilephone System on Chips.

Although such a heterogeneous design helps in energy efficiency, it makes scheduling decisions more complex. The main difficulty is that to minimize the power consumption, sometimes it is better to schedule a task on the performance core if it is compute intensive and is going to run for long. However, we don't know how long a process will execute up front. Schedulers have to make a choice based on past information.

For this assignment, we have provided you with a simulator that models a simplified heterogeneous processor configuration. You are expected to fill the epoch function in scheduler.cpp and submit a single cpp file. You can change the rest of the simulator for debugging purposes, but your scheduler.cpp code should run without any modifications to the other files.

Simplifying assumptions

For the purpose of this assignment, we have made the following simplifications:

- A process can be abstracted as a series of bursts that are either CPU burst or IO Burst. Since the CPU burst involve executing instructions on the CPU core, the time take to complete a CPU burst will vary depending on whether it was scheduled on a big or Little core. But the time taken to complete an IO burst will be the same irrespective of the core scheduled on.
- For every core, we specify the speed of the processor in MIPS (Millions of Instructions per Second). There is no Hyperthreading and no Dynamic Voltage Frequency Scaling (so only one fixed frequency) on the cores. All CPU core consumes 0 power when turned off, consume a specified idle power when turned on but not executing any process, and consumes a specified running power when turned on and executing a process. The CPU cores can be powered on or off instantaneously with no delay.
- Although the simulator takes core configuration as input. We will evaluate only on the given configuration:
  - 4 big cores running at 4000 MIPS, 4 W Idle power and 8 W running power
  - 4 Little cores running at 1000 MIPS, 2 W Idle power and 4 W running power
- The scheduler is **Cooperative**, i.e. there will be no preemption, i.e. once a CPU burst is scheduled on a core, it will run to completion. The only exception is when a process is migrated from one core to another, in which case the CPU burst will be paused and resumed on the new core.
- All processes have the same priority. Once a process is scheduler on a Core, the Core uses a

First-Come First-Serve policy (breaking ties with PID) for selecting a CPU burst among the ready processes scheduled on it.

- To make things simple, the scheduler is only invoked at fixed time intervals called Epochs (100 ms). Processes arrive only at epoch boundary. At an epoch boundary, the scheduler can decide to power-on or power-off a core, schedule a process on a particular core and also migrate a process from one core to another. The scheduler can use information about the past CPU burst of processes, as well as the energy and load on the cores during the previous epochs.
- When a process is migrated to another core, there will be a fixed time performance penalty to simulate cache misses on the new core.
- We will use Energy Delay Product (EDP) as the metric to evaluate the scheduler. We define EDP of a process as the product of energy consumed and the turn around time of a process. It is a measure of both performance and energy efficiency. The lower the EDP value, the better the scheduling decisions.

Part 1: Three scheduling approaches (7 marks)

In the first part of this assignment, your objective is to implement the following schedulers which use three different views of heterogeneous architecture, and compare them based on Energy Delay Product (EDP). In all the three schedulers we follow a **Round-Robin** to assign processes to cores, i.e. the first process that arrives is scheduled on the first core, second on second, and so on. If there are only 4 cores in use, then the fifth arriving process is scheduled again on the first core.

1. Clustering scheduler
    - In this scheme, we view the big and Little cores forming two different clusters. At any point of time, only one cluster (consisting of 4 cores) is active. Initially, the scheduler uses the the four cores belonging to the Little core cluster.
    - At an epoch boundary, if the average load during last epoch on all cores of the Little cluster is greater than or equal to 50%, we switch to the big cluster, migrating all processes to the corresponding core in the big cluster.
    - At an epoch boundary, if the average load during last epoch on all cores of the big cluster is lesser than 50%, we switch to the Little cluster, migrating all processes to the corresponding core in the Little cluster.
2. Switcher scheduler
    - In this scheme, Little cores 0-3 are paired up with big cores 4-7 to form four virtual cores. For instance, core 1 is paired with core 5.
    - At an epoch boundary, if the average load during last epoch on a Little core is greater than or equal to 50%, the virtual core is switched to use the corresponding big core. The processes scheduled on the Little core is migrated to the corresponding big core.
    - At an epoch boundary, if the average load during last epoch on a big core is lesser than

50%, the virtual core is switched to use the corresponding Little core. The processes scheduled on the big core is migrated to the corresponding Little core.

3. Global Task scheduler
    - In this scheme, there is no process migration. All eight cores are active at all times, and the processes are assigned in Round-Robin across all 8 cores.

For more details about the three approaches, refer https://en.wikipedia.org/wiki/ARM_big.LITTLE. The evaluation for this part will be testcase based. We have currently provided only a sample testcase. We will share some actual testcases later.

Part 2: Your own Scheduler !! (8 marks)

In the second part of this assignment, you are expected to create your own scheduler, (we will call it Bonus scheduler) that optimizes for Energy Delay Product, i.e. it tries to achieve performance while still being energy efficient.

Note that in the scheduler, you are not allowed to use future information of processes. For instance, you cannot iterate over bursts that have not arrived at the core to make a decision. Even for a burst that has arrived, you cannot use the burst length to make a decision. Use only past information, i.e. statistics about bursts that have already completed.

You can develop and evaluate multiple schedulers to test out different ideas. But at the end, you must submit only one scheduler that performs best on EDP. You can give a description and metrics of the other schedulers in your report.

The evaluation for this part will be based on the code and the report. Ensure that your code is easy to understand. Use meaningful variable names and add comments where necessary. In your report, include a description of the bonus scheduler that you have implemented. Describe the intuition and rationale behind the design and the results of any sensitivity study that you have performed to select the parameters for the scheduler. Compare your scheduler with the three schedulers implemented in Part 1. Also mention the type of workloads for which the different schedulers perform best.

Input format

The input format of the simulator is as follows. It consists of a core list followed by a process list.

- The core list starts with the number of cores in the system, followed by one line for each core. The line consists of the speed of the core in MIPS (Millions Of Instructions per Second), the power consumed by the core when it is idle (powered on but not executing) and the power consumed by the core when it is running (executing a process).
- The process list consists of the number of processes, followed by the one line for each process. The line consists of the arrival time of the process, the number of bursts, followed by that many number of burst values. CPU bursts specify the number of instruction in millions to

be executed and the IO burst specify the time taken for IO. The process list will be sorted according to arrival time

```
<num_cores>
<MIPS> <idle_power> <running_power>
<MIPS> <idle_power> <running_power>
...
<num_processes>
<arrival_time> <num_bursts> [<cpu_instructions_in_millions> <io_wait_time>]+
<arrival_time> <num_bursts> [<cpu_instructions_in_millions> <io_wait_time>]+
...
```

Submission guidelines

- Submit only two files: The scheduler.cpp file renamed as <ROLLNO>.cpp, along with the report named as <ROLLNO>.pdf.

References

- Wikipedia article on big.Little architecture
- LWN post on Big.Little Scheduling